

DOCKER -

• containers - isolated env to run an app.

• container registry - private or public

• dockerhub

• simplifying dev process

• pulling images

• `docker pull <img-name>`

• `docker run <name>`

• `docker ps`

• img vs container diff

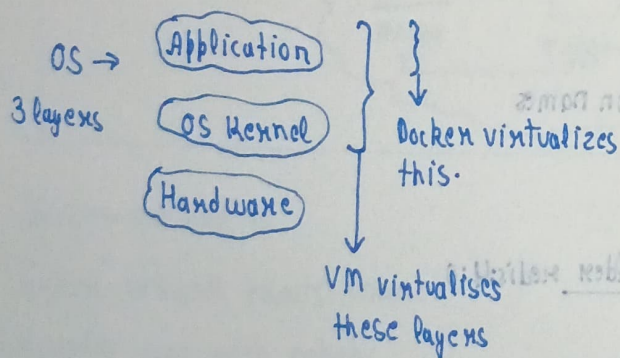
↓
contain all applicatⁿ,
dependency ..., portable

when img pulled on machine & run,
creates container env. | Running env for img

• can have diff ver of same apps at same time.

• host port & container port

• Docker vs VM (Both Virtualizatiⁿ tool)



VM Img	Docker Img
• Large	• Small
• Slower to start & run	• Run, start fast
• VM img of any ^{os} host can run on any OS host	• A linux-based container img may not be compatible on another host like Windows.

Appⁿ
Window Kernel H/W
Linux Docker img
might not be compatible with window kernel.

So we need docker toolbox.

Abstracts away kernel to make
psbl for kernel to run diff host img.

<id> * containers also have own names

<name>

```
-d -p 6000:6379 --name redis-oldest redis:4:0
```

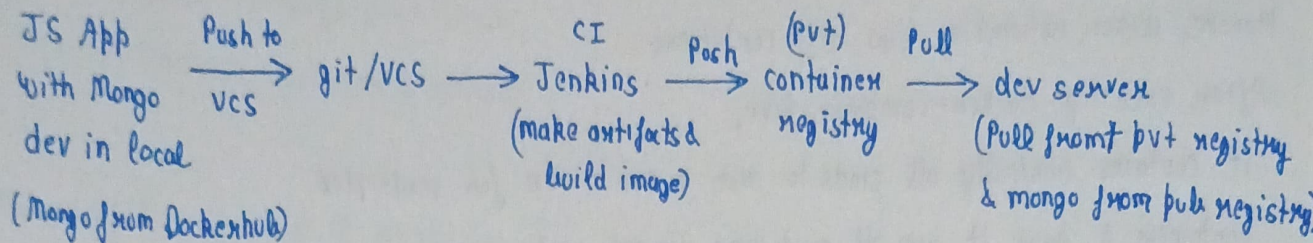
ec - get terminal of running container

-it <id> /bin/bash
<name>

n vs docker start

8	work on container
containers	start/stop containers

CI Workflow

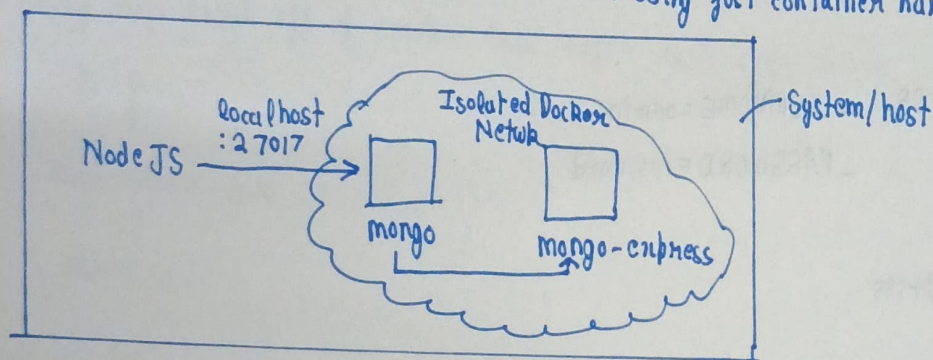


Docker in dev-

We use mongo
mongo-express UI/images

We also need to connect these two images: using docker network.

Running several containers in same isolated docker network:
containers can communicate with each other using just container names.



docker networks

docker network create <name>

running cont. with network:

```

→ docker run -p 27017:27017 -d -e MONGO_INITDB_ROOT_USERNAME=admin mongo
    --name mongodb --net mongo-network <name>
    ↑
    attach to network
  
```

```

→ docker run -d -p 8081:8081 -e ME_CONFIG_MONGODB_ADMINUSERNAME=admin
    --net mongo-network --name mongo-express -e <other-env-var> mongo-express
  
```

mongo-express will connect to mongodb.

then using our nodejs app we can exchange data with db.

• Docker-compose - yaml

Makes running multiple containers process easy & automated by using yaml config.

Handles docker networking b/w diff containers.

After creating docker-compose file,

it contains basically all cmds to run a container for multiple containers & have it run all containers in one cmd.

→ Basic compose yaml to start up 2 containers,

version: '3'

services:

mongodb:

image: mongo

ports:

- 27017:27017

environment:

- MONGO_INITDB_ROOT_USERNAME=admin

- PASSWORD=password

mongo-express:

image: mongo-express

ports:

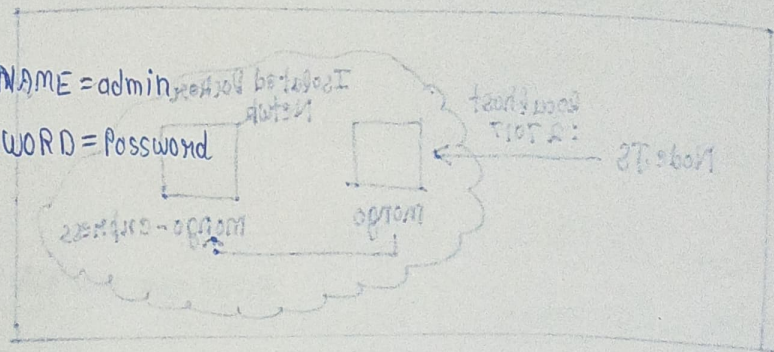
- 8080:8081

environment:

- ME_CONFIG_MONGODB_USERNAME=admin

- ME_CONFIG_MONGODB_PASSWORD=password

- ME_CONFIG_MONGODB_SERVER=mongodb



• docker-compose -f mongo.yaml up

→ creates its own network in which all containers of compose live.

- When normally running containers, restarting them, lose data.

- `docker-compose -f mongo.yml down`
removes container & network.

→ Dockerfile - Blueprint of docker img

Make/dev an app, package into its own container, & easy deployment.

cmds like FROM, ENV, RUN, COPY, CMD

- * RUN - exec any linux cmd on container / not host.

RUN `cp ./home/app` → exec on container entirely

COPY `./home/app` → exec on host to copy in container } *

- * Every image based off of another base image - Multi Layered images

app:1.0
↑

node: alpine-3.10
↑

alpine-3.10

→ Dockerfile - must be this name

- Build image →

`docker build -t my-app:version-1 .` ↙ Doc of Dockerfile

→ now just run image in container using `docker run`.

- `docker rm <id>` - remove container uses container-id

- `docker rmi <id>` - remove image uses img-id

- `docker exec -it <id> /bin/sh`
 `/bin/bash` } one of these will always work.

- optimise Dockerfile by only copying needed files

• Private Docker Registry -

AWS Elastic Container Reg

• docker login

• aws cli

• documentation

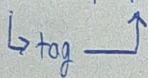
• Naming in registry -

registry Domain / imageName: tag

• docker push <img-name>:tag

• docker tag <img-name> <repo>:tag

• make change → rebuild → push to AWS again



→ Deploy -

Log in to dev server

Part image pulled from put registry & public img from public registry.

Have a compose.yml with put & public images

Server logs in to pull from put reg.

Compose file used on server to deploy all containers.

• Have compose on server anyhow, just up it, & app is live. ★

→ Volumes -

Offers data persistence

A file/folder of phy. file system is plugged into virtual file system.

3 Types -

① docker run

-v /home/mount/data : /var/lib/mysql/data

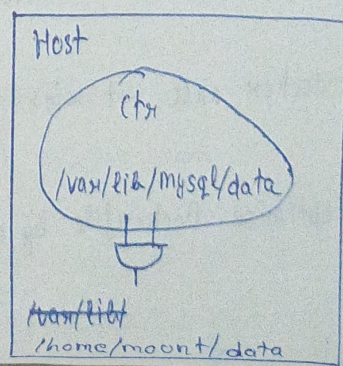
```
graph LR; host["/home/mount/data"] --- x((X)); x --- container["/var/lib/mysql/data"]
```

host directory

container directory

* we decide where on host file sys, ref is made.

(Host Volumes)

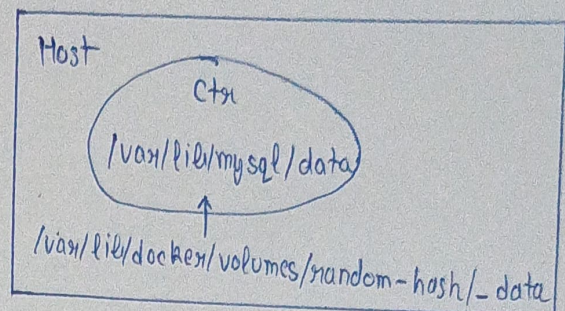


② docker run

(Anonymous Volumes)

-v /var/lib/mysql/data
← only container disc →

* for each container a folder is gen. that gets mounted



③ docker run

(Named Volumes)

-v <name>:/var/lib/mysql/data
← volume name × ctr data →

same

* can reference volume by name.

* should be used in prod.

→ Using volumes, data persists, & container restart, does not remove data